



C#-OOPII

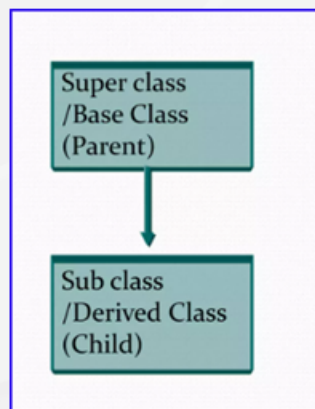
- Inheritance
- Constructor hierarchy
- Polymorphism
- Method Overloading
- Method Overriding
- Hiding, using new

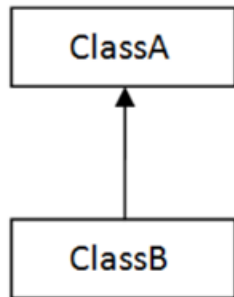


- Inheritance is the capability of a class to use the properties and methods of another class while adding its own functionality.
- Enables you to add new features and functionality to an existing class without modifying the existing class.

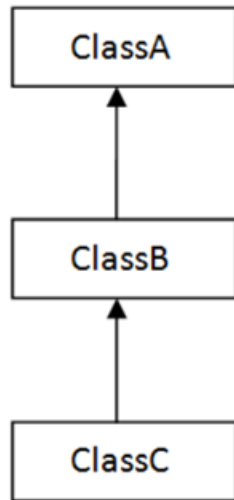
Base class and Subclass

- A Baseclass or parent class is the one from which another class inherits attributes and behavior.
- A subclass or child class is a class that inherits attributes and behavior from a Baseclass.

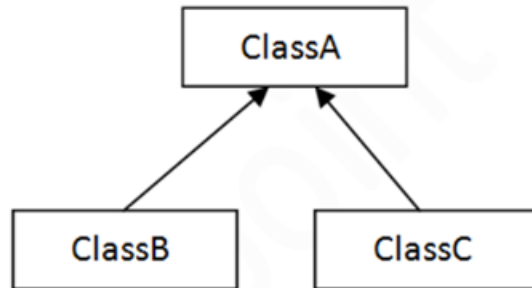




1) Single



2) Multilevel



3) Hierarchical

The hierarchy of classes A, B, C

```
class A
{
    // Class A constructor
    public A() { }
}
```

```
class B : A
{
    // Class B constructor
    public B() { }
}
```

```
class C : B
{
    // Class C constructor
    public C() { }
}
```

Using class instances, calling constructors

```
// Creating an instance of class A,
// the constructor A() is called
A objA = new A();
```

```
// Creating an instance of class B,
// constructors call sequence:
// A() => B()
B objB = new B();
```

```
// Creating an instance of class C,
// constructors call sequence:
// A() => B() => C()
C objC = new C();
```

■ Important Points:

- In C#, three types can participate in inheritance: Class, Struct, and Interface.
- A class can inherit a single class only. It cannot inherit from multiple classes.
- A class cannot inherit from a struct.
- A class can inherit (implement) one or more interfaces.
- A Struct can inherit from one or more interfaces. However, it cannot inherit from another struct or class.
- An interface can inherit from one or more interfaces but cannot inherit from a class or a struct.
- Constructors or destructors cannot be inherited.

Constructor hierarchy

Order of constructors calling with inheritance

- **With non-static constructors**
 1. In case of inheritance if we create an object of child class then the parent class constructor will be called before child class constructor.
 2. i.e. The constructor calling order will be top to bottom.

```
1 using System;
2 public class Person
3 {
4     public Person()
5     {
6         Console.WriteLine("I am Person constructor.");
7     }
8 }
9 public class Student : Person
10 {
11     public Student()
12     {
13         Console.WriteLine("I am Student constructor.");
14     }
15 }
16 class Program
17 {
18     static void Main(string[] args)
19     {
20         Student st = new Student();
21         Console.Read();
22     }
23 }
```

C:\Snehal\CDAC\Sept 22\DAC x + v

I am Person constructor.
I am Student constructor.

With static constructors

- In the case of a static constructor, calling order will be reversed from non-static i.e. it will call from **bottom to top**.

```
using System;
public class Person
{
    static Person()
    {
        Console.WriteLine("I am Person constructor.");
    }
}
public class Student : Person
{
    static Student()
    {
        Console.WriteLine("I am Student constructor.");
    }
}
class Program
{
    static void Main(string[] args)
    {
        Student st = new Student();
        Console.Read();
    }
}
```

C:\Snehal\CDAC\Sept 22\DAC X + v

```
I am Student constructor.
I am Person constructor.
```

- Polymorphism is a Greek word that means multiple forms or shapes.
- You can use polymorphism if you want to have multiple forms of one or more methods of a class with the same name.
- In C#, polymorphism can be achieved in two ways:
 1. **Compile-time Polymorphism** (Method Overloading)
 2. **Run-time Polymorphism**

- Compile-time polymorphism is also known as method overloading. C# allows us to define more than one method with the same name but with different signatures. This is called method overloading.
- Method overloading is also known as early binding or static binding because which method to call is decided at compile time, early than the runtime.
- **Rules for Method Overloading:**
 1. Method names should be the same but method signatures must be different. Either the number of parameters, type of parameters, or order of parameters must be different.
 2. The return type of the methods does not play any role in the method overloading.
 3. Optional Parameters take precedence over implicit type conversion when deciding which method definition to bind.

```
class ConsolePrinter
{
    public void Print(string str)
    {
        Console.WriteLine(str);
    }

    public void Print(string str1, string str2)
    {
        Console.WriteLine($"{str1}, {str2}");
    }

    public void Print(string str1, string str2, string str3)
    {
        Console.WriteLine($"{str1}, {str2}, {str3}");
    }

    public void Print(int a)
    {
        Console.WriteLine($"Integer {a}");
    }

    public void Print(int a, string str)
    {
        Console.WriteLine($"{a}, {str}");
    }

    public void Print(string str, int a)
    {
        Console.WriteLine($"{a}, {str}");
    }
}
```

- Run-time polymorphism is also known as inheritance-based polymorphism or method overriding.
- Inheritance allows you to inherit a base class into a derived class and all the public members of the base class automatically become members of the derived class.
- However, you can redefine the base class's member in the derived class to provide a different implementation than the base class. This is called method overriding that also known as runtime polymorphism.
- In C#, by default, all the members of a class are sealed and cannot be redefined in the derived class. Use the **virtual** keyword with a member of the base class to make it **overridable**, and use the **override** keyword in the derived class to indicate that this member of the base class is being redefined in the derived class

```
class Person
{
    public virtual void Greet()
    {
        Console.WriteLine("Hi! I am a person.");
    }
}

class Employee : Person
{
    public override void Greet()
    {
        Console.WriteLine("Hello! I am an employee.");
    }
}
```

```
class Program
{
    public static void Main()
    {
        Person p1 = new Person();
        p1.Greet();

        Person p2 = new Employee();
        p2.Greet();

        Employee emp = new Employee();
        emp.Greet();
    }
}
```

Microsoft Visual Studio Debug Console

```
Hi! I am a person.
Hello! I am an employee.
Hello! I am an employee.
```

- A method, property, indexer, or event can be overridden in the derived class.
- Static methods cannot be overridden.
- Must use virtual keyword in the base class methods to indicate that the methods can be overridden.
- Must use the override keyword in the derived class to override the base class method.

- Use the new keyword in the derived class to hide the base class method. This will be useful when you are resuing third-party APIs where you don't have control over the base class.

```
class Person
{
    public virtual void Greet()
    {
        Console.WriteLine("Hi! I am a person.");
    }
}

class Employee : Person
{
    public new void Greet()
    {
        Console.WriteLine("Hello! I am an employee.");
    }
}
```

```
class Program
{
    public static void Main()
    {
        Person p1 = new Person();
        p1.Greet();

        Person p2 = new Employee();
        p2.Greet();

        Employee emp = new Employee();
        emp.Greet();
    }
}
```

Microsoft Visual Studio Debug Console

```
Hi! I am a person.
Hi! I am a person.
Hello! I am an employee.
```

- The "override" modifier extends the base class method, and the "new" modifier hides it.
- The "virtual" keyword modifies a method, property, indexer, or event declared in the base class and allows it to be overridden in the derived class.
- The "override" keyword extends or modifies a virtual/abstract method, property, indexer, or event of the base class into the derived class.
- The "new" keyword is used to hide a method, property, indexer, or event of the base class into the derived class.
- If a method is not overriding the derived method then it is hiding it. A hiding method must be declared using the new keyword.
- Shadowing is another commonly used term for hiding. The C# specification only uses "hiding" but either is acceptable. Shadowing is a VB concept.

- For hiding the base class method from derived class simply declare the derived class method with the new keyword.
Whereas in C#, for overriding the base class method in a derived class, you need to declare the base class method as virtual and the derived class method as overridden.
- If a method is simply hidden then the implementation to call is based on the compile-time type of the argument "this".
Whereas if a method is overridden then the implementation to be called is based on the run-time type of the argument "this".
- New is reference-type specific, overriding is object-type specific.

- Inheritance
- Constructor hierarchy
- Polymorphism
- Method Overloading
- Method Overriding
- Hiding, using new



Thank You